CS224R Spring 2026 Homework 2
Online Reinforcement Learning

Due 4/24/2026

SUNet ID:
Name:
Collaborators:

By turning in this assignment, I agree by the Stanford honor code and declare that all of this is my own work.

Overview

Goals: In this assignment, you you will implement Q-learning on a gridworld and implement RL algorithms to solve a realistic robot task, in which the agent needs to pick up a tool and use it to hammer a nail. The agent controls a 4 degree of freedom Sawyer robot with a continuous action space and receives environment states. The degrees of freedom refers to the dimensions of the actions provided to the robot. Providing dense rewards for real world problems is difficult, so in this environment the agent receives a sparse reward of 1.0 upon fully completing the task and no intermediate rewards.

The implementations will be split into three parts. In the first part, you will implement a Q-Learning algorithm on a gridworld to test out the effects of different reward functions. In the second part, you will implement an on-policy algorithm. In the third part, you will implement an off-policy algorithm. This homework is intended to allow you to explore some of the design choices of online RL algorithms and how they affect performance.

Setup

For this assignment, you will complete all sections on Modal instances.
We recommend only using Modal for compute, as we **do not** support setup on any other platform, such as your own local computer. You may still develop code on your local computer, which we suggest to save compute credits. You may find helpful this guide on how to set up your code to run on modal.

For simplicity, for this homework we already provide you with the entry points to run on modal, so you won't have to worry about setting it up yourself, but this guide will be handy for the homework. When running the experiments, the computer terminal needs to keep running otherwise the modal job will crash.

Start by creating a conda environment that you will use to launch modal with from your local machine.

```
conda env create -f conda_env_modal.yml
conda activate cs224r-hw2
```

We also provide a conda environment for running your code locally in the case that you have a linux machine with an nvidia GPU. However, this is just a reference for those that dare to give it a shot and won't be supported by the teaching staff.

```
conda env create -f conda_env_local.yml
conda activate cs224r-hw2-local
```

Weights and Biases Experiment Tracking: This homework uses Weights and Biases (Wandb) logging to track training stats. You will need a wandb account at https://wandb.ai/site/.

Step 1 — Create a wandb account
Go to https://wandb.ai/site and sign up for a free account. You can authenticate using your existing GitHub or Google credentials.

Step 2 — Install the wandb library
Install via `conda` in your terminal (or add it to your `environment.yml`):
`conda install -c conda-forge wandb`
This step should be handled automatically when installing the conda environment.

Step 3 — Authenticate with your API key
Run the login command and paste your API key when prompted:
`wandb login`
Your API key can be found at https://wandb.ai/authorize. The key is saved locally, so you only need to run this once per machine.

If you have any issues with the wandb key not found when running on modal try running the following:

```
modal secret create wandb-secret WANDB_API_KEY=<wandb_key> --force
```

Viewing your results
Once training starts, wandb prints a direct URL to your terminal. Open it to see your metrics update in real time or you can also go to https://wandb.ai/home and click into the project.

Submitting the PDF, Code, and Runs: Make a PDF report containing: observed outcomes from Question 1, training curves from Question 2, and 3, and your responses.

Submit the **grid_world_q_learning.py**, **on_policy.py** and **off_policy.py** file and the csv downloaded from Wandb for the final runs.

To download the csv file, go to "Charts" on your wandb run, go to eval/episode_success section and click on the three dots on the top right corner as shown in the picture below:

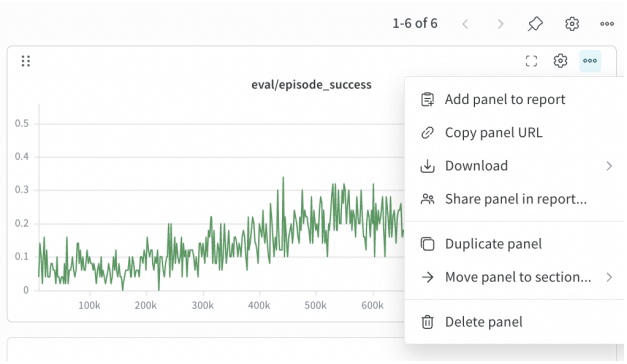

Figure 1: CSV file download instruction.

Gradescope: Submit both the PDF and the zipped code and experiment runs in the appropriate assignment on Gradescope.

Use of AI Tools (e.g. ChatGPT, Cursor) For the sake of deeper understanding on implementing actor-critic methods, assistance from generative models to write code for this homework is prohibited. See the course website for more details.

Sample File Submission

For this part you should submit the **gridworld_q_learning.py**, **on_policy.py** and **off_policy.py** file, as well as the logs from your final three runs, which are saved in the logging folder. Your submission file should look like this

```
submit.zip
├─ on_policy.py
├─ off_policy.py
└─ CSV_files
   ├─ on_policy.csv
   ├─ off_policy.num_critics=2,utd=1.csv
   └─ off_policy.num_critics=10,utd=5.csv
```

We advise you to start as early as possible since the assignment requires longer compute times.

Problem 1: [3 points]

Your Tasks

For each scenario, report the whether it reaches a goal, and if so which one it reaches. Give a one sentence explanation for why it learned that trajectory in each scenario.

Problem 2: [3 points]

Attach a screenshot of the plot `eval/episode_success` for this run up to 1 million steps from Wandb.

Problem 3: [3 points]

Attach a screenshot of the plot "eval/episode_success" for the off-policy run from Wandb up to 100k steps. You should achieve a success rate of at least 90% before 100K environment steps.

Attach a screenshot of the plot "eval/episode_success" for the off-policy UTD 5 run up to 40k steps and compare it to the previous question. Provide a brief, one-sentence explanation of why we observe these effects.

Compare the PPO learning curve (`eval/episode_success`) from Problem 2 with the actor-critic curve from Problem 3 (2 critics, UTD=1).

In 3–5 sentences, explain at least two concrete differences you observe between the two algorithms in terms of sample efficiency and final performance. Justify each observed difference by connecting it to a specific property of each algorithm.